UD02
INTRODUCCIÓN A JAVA

Francisco Lázaro Pérez Vidal

fl.perezvidal@edu.gva.es

0.INDICE

1. Variables
2. Literales
3. Operadores
4. Conversiones de tipo
5. Comentarios
6. Organización de ficheros
7. Entrada desde teclado
8. Salida a pantalla

1.VARIABLES

Tipos de datos

Para declarar una variable, es necesario antes conocer su tipo. En Java tenemos principalmente los siguientes tipos de datos:

Tipo	Definición
byte	Entero muy corto (-128 a 127)
short	Entero corto
int	Entero
long	Entero largo
float	Real con 7 cifras decimales de precisión
double	Real con 16 cifras decimales de precisión
char	Carácter (1 dígito)
boolean	Lógico (verdadero/falso)

1.VARIABLES

Declaración e inicialización

Una vez conocido el tipo de dato, podemos declarar la variable de la siguiente forma:

```
int edad;
```

Podemos declararla e inicializarla así:

```
int edad = 18;
```

También podemos definir una constante, es decir, aquella cuyo valor no podemos cambiar a lo largo del programa. Para ello, utilizaremos la palabra reservada “final” y el nombre de la constante en mayúsculas (por convención).

```
final int IVA = 21;
```

```
Final int iva = 21; //mal visto
```

1.VARIABLES

Declaración e inicialización

Las declaraciones de variables pueden ir en cualquier parte del programa, pero siempre antes de que la variable sea usada.

Por convención, se declaran todas arriba, de esa forma el scope (el ámbito de la variable) son todas las líneas de código del programa.

Dependiendo del tipo de dato, se inicializan a sus valores correspondientes, p.ej.:

```
int temperatura = -18;

final double PI = 3.14159;

char letra = ‘a’;

boolean clienteMoroso = true;
```

2.LITERALES

char y String

En Java tenemos la posibilidad de expresar literales de carácter y de cadenas de caracteres, veamos la diferencia:

Tipo	Definición	Ejemplos
char	1 carácter encerrado entre comillas simples	`char separador = '@';` `char respuesta = 'A';`
String	1 o más caracteres encerrados entre comillas dobles	`String nombre = "Paco Pérez";` `String respuesta = "B";`

Además, disponemos de caracteres especiales que pueden expresar tabulación, salto de línea o la impresión de una doble comilla dentro de una cadena de texto.

2.LITERALES

Ejemplos de caracteres especiales

Caracter	Definición	Ejemplos
“\t”	Tabulador	**String cuenta = “Uno”+”\t”+”Dos”+”\t”+Tres”;** Uno Dos Tres **String cuenta2 = “Cuatro\tCinco\tSeis”;** Cuatro Cinco Seis
“\n”	Salto de línea	**String cuenta = “Uno\nDos”;** Uno Dos
\”	Comillas dobles	**String saludo = “Paco dijo \”Hola\” con energía”;** Paco dijo “Hola” con energía

3.OPERADORES

Operadores de asignación

El principal es ‘=‘ pero también hay más operadores de asignación:

Operador	Versión compacta	Versión desarrollada

+=	`op1 += op2`	`op1 = op1 + op2`
		
		P.ej.: `int cuenta = 10`
		`cuenta+=3`
		¿Qué valor almacena la variable *cuenta*?

-=	`op1 -= op2`	`op1 = op1 - op2`
*=	`op1 *= op2`	`Op1 = op1 * op2`
/=	`op1 /= op2`	`op1 = op1 / op2`
%=	`op1 %= op2`	`op1 = op1 % op2`

3.OPERADORES

Operadores lógicos

Nos permiten construir expresiones lógicas

Operador	Definición
&&	"Y" lógico. Devuelve TRUE si ambos operandos son TRUE.
||	"O" lógico. Devuelve TRUE si alguno de los operandos es TRUE.
!	Negación lógica. Niega la expresión que se le pasa

3.OPERADORES

Operadores lógicos. Ejemplo.

```
boolean tieneCarnet = true;
boolean llevaCasco = false;

IMPRIMIR ("¿Puede conducir?");
IMPRIMIR (tieneCarnet && llevaCasco); // false

IMPRIMIR ("¿Le falta algo?");
IMPRIMIR (!(tieneCarnet && llevaCasco)); // true (Expresión anterior negada)

IMPRIMIR ("¿Cumple al menos un requisito?");
IMPRIMIR (tieneCarnet || llevaCasco); // true
```

Por sencillez y para centrarnos en el funcionamiento de los operadores lógicos, mantenemos el término algorítmico de IMPRIMIR, cuya forma en Java se estudiará más adelante.



3.OPERADORES

Operador de concatenación

Concatenamos con el símbolo “+” aplicado a cadenas de texto

```
String nombre = “Paco”
IMPRIMIR “¡Hola ”+nombre+”!” //Imprime “Hola Paco!”
```

Operadores incrementales (++) o decrementales (--)

Nos permiten incrementar o decrementar las variables en 1 unidad prefija o sufija.

```
int contador = 5
contador++
IMPRIMIR contador //Imprime un 6
contador--
IMPRIMIR contador //Imprime un 5
```

3.OPERADORES

Operadores incrementales (++) o decrementales (--). Ejemplo

Código	Traza de x
`int x = 10;`	x=10
`IMPRIMIR x;`	**10**
`x++;`	Incrementa en 1. x=11
`IMPRIMIR x;`	**11**
`IMPRIMIR ++x;`	Se incrementa a 12 y luego se imprime. x = **12**
`IMPRIMIR x++;`	Se imprime **12** y luego se incrementa. x = 13
`IMPRIMIR x;`	**13**
`IMPRIMIR x++;`	Se imprime **13** y luego se incrementa. x = 14
`IMPRIMIR ++x;`	Se incrementa a **15** y luego se imprime. x=15

4.CONVERSIONES DE TIPO

Casting implícito o automático

Cuando a una variable de un tipo se le asigna un valor de otro tipo numérico con menos bits para su representación, se realiza una conversión automática. Dicho de otra forma, un valor de un tipo pequeño siempre se va a poder almacenar en una variable de un tipo más grande. Los tipos de datos son distintos, pero la asignación se lleva a cabo correctamente.

En este caso no se necesita escribir código para que la conversión se lleve a cabo. Ocurre cuando se realiza lo que se llama una conversión ancha (*widening casting*), es decir, cuando se coloca un valor pequeño en un contenedor grande.

Ejemplo:

```
int num1 = 100;

long num2 = num1;  //un int “cabe” en un long
```

4.CONVERSIONES DE TIPO

Casting explícito

Ocurre cuando hacemos una conversión de un tipo con más bits a un tipo con menos bits. Es decir, cuando se coloca un valor grande en un contenedor pequeño.

En estos casos, debemos indicar que queremos hacer la conversión de manera expresa, ya que se puede producir una pérdida de datos y hemos de ser conscientes de ello. Este tipo de conversiones se realiza con el operador cast. El formato del operador es el siguiente:

```
(tipo)valor a convertir
```

Ejemplo:

```
int num1   = 100;

short num2 = (short) num1;  //short tiene menor rango que int
```

4.CONVERSIONES DE TIPO

Casting explícito. Ejemplo

Debemos tener en cuenta que un valor numérico nunca puede ser asignado a una variable de un tipo menor en rango, si no es con una conversión explícita. P. ej.:

```
int a;

byte b;


a = 12; // no se realiza conversión alguna

b = 12; // se permite porque 12 está dentro del rango permitido de valores para b

b = a;  // error, no permitido (incluso aunque 12 podría almacenarse en un byte)


b = (byte) a; // Correcto, forzamos conversión explícita
```

4.CONVERSIONES DE TIPO

Conversión de números reales (float, double) a enteros (int)

Cuando convertimos números en coma flotante a números enteros, la parte decimal se trunca (redondeo a cero). Si queremos hacer otro tipo de redondeo, podemos utilizar, entre otras, las siguientes funciones:

`Math.round(num)`: Redondeo al siguiente número entero.

`Math.floor(num)`: Redondeo al numero actual.

Por ejemplo:

```
double num = 3.5;

x = Math.round(num);  // x = 4

z = Math.floor(num);  // z = 3
```

4.CONVERSIONES DE TIPO

Conversión de tipo con cadenas de caracteres (String)

Para convertir una cadena de texto `cad` a otros tipos de datos se utilizan las siguientes funciones:

```
num = Byte.parseByte(cad);

num = Short.parseShort(cad);

num = Integer.parseInt(cad);

num = Long.parseLong(cad);

num = Float.parseFloat(cad);

num = Double.parseDouble(cad);
```

Por ejemplo, si hemos leído de teclado un número que está almacenado en una variable de tipo String llamada cadena, y lo queremos convertir al tipo de datos int, haríamos lo siguiente:

```
int num = Integer.parseInt(cadena);
```
//leemos, p ej. “12” y almaceno el número entero 12

5.COMENTARIOS

Comentarios de una sola línea

Utilizaremos el delimitador // para introducir comentarios de sólo una línea.

```
// comentario de una sola línea
```

Comentarios de varias líneas

Para introducir este tipo de comentarios, utilizaremos una barra inclinada y un asterisco (/*), al principio del párrafo y un asterisco seguido de una barra inclinada (*/) al final del mismo.

```
/* Esto es un comentario
 * de varias líneas.
 * En concreto, de 3 líneas. */
```

6.ORGANIZACIÓN DE FICHEROS

Extensión de los ficheros

El programa o código fuente se codificará en un fichero con extensión `.java` que, al compilarse, dará como resultado un fichero con extensión `.class`

Package e import

Dentro del IDE, será posible repartir los programas de nuestro proyecto dentro de distintos paquetes software. Por defecto, trabajaremos en el paquete software por defecto o "`default package`". De no ser así, se especificaría el nombre del paquete con el formato `package nombrePaquete`

Por otro lado, es posible que necesitemos en nuestro programa de paquetes software que ya tiene desarrollados java, para lo que utilizaríamos el formato `import nombrePaquete`

```
P.ej:  package altaUsuarios;

       import java.util.ArrayList;
```

7.ENTRADA DESDE TECLADO

La clase Scanner

La clase Scanner de Java provee distintos métodos para leer valores de entrada de varios tipos y está localizada en el paquete *java.util.* Así que, para facilitarnos la tarea en el aula, es altamente recomendable comenzar nuestro código con la siguiente línea:

```
import java.util.*;
```

Dentro de nuestro código fuente, cuando declaremos las variables, crearemos el Scanner así:

```
Scanner s = new Scanner(System.in)
```

De esta forma, dentro de la variable “s” se referencia a una zona de memoria que recogerá la información de la entrada estándar del sistema (el teclado).

Una vez realizada la importación del paquete y declarado el Scanner, podremos comenzar a leer datos desde teclado, utilizando un método distinto en función del tipo de dato que se vaya a leer.

7.ENTRADA DESDE TECLADO

Métodos para leer

Primero, declaramos el Scanner con la siguiente línea:

```
Scanner s = new Scanner(System.in)
```

Luego, podremos utilizar los siguientes métodos para leer desde teclado:

Método	Definición
`String palabra = s.next();`	Lee hasta el siguiente espacio
`String frase = s.nextLine();`	Lee hasta el final de la línea (“\n”)
`int numero = s.nextInt();`	Lee un número entero
`double altura = s.nextDouble();`	Lee un número real
`Long calculo = s.nextLong();`	Lee un entero largo (long)
`Boolean decisión = s.nextBoolean();`	Lee un valor lógico (true/false)

7.ENTRADA DESDE TECLADO

Bug de Java de lectura

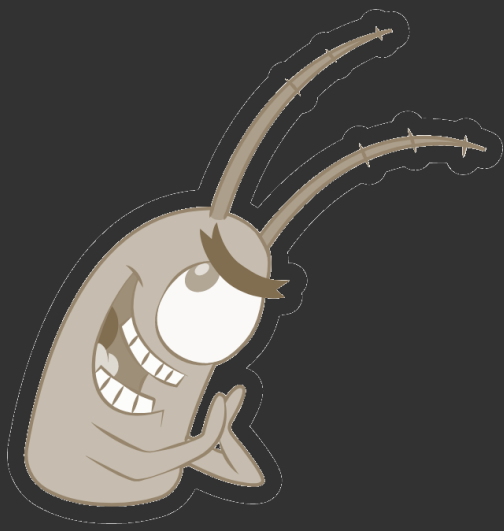
```
Scanner sc = new Scanner(System.in);
int numClase;
String nombre;
double nota;

IMPRIMIR “Introduce el número de clase:”;
numClase = sc.nextInt();   //P ej. 5”\n”

//NOTA: esta línea es para capturar el retorno de carro
sc.nextLine();

IMPRIMIR “Introduce el nombre del alumno:”;
nombre = sc.nextLine();

IMPRIMIR “Introduce la nota del examen:”;
nota = sc.nextDouble();
```



8.SALIDA A PANTALLA

Métodos con salto de línea y sin salto de línea

Ya no utilizaremos más la forma “IMPRIMIR” de los algoritmos, sino que en su lugar utilizaremos los siguientes métodos de Java para imprimir por pantalla:

Método	Definición
System.out.println(“Hola”) **System.out.println(“Adios”)**	Cada vez que imprime, imprime también un salto de línea (“\n”): Hola Adios
System.out.print(“Hola”) **System.out.print(“Adios\n”)** **System.out.print(“Le pido”)**	Imprime exactamente el valor facilitado entre paréntesis: HolaAdios Le pido

8.SALIDA A PANTALLA

Método para salida formateada

Utilizamos una plantilla donde se acomodarán los datos uno a uno:

%s cadena de texto
%d entero
%f real
%c carácter

```
String nombre = "Paco";
int edad = 46;
double altura = 1.8256;
char metros = 'm';

System.out.printf("Hola me llamo %s y tengo %d años\n", nombre, edad);
System.out.printf("La estatura de %S es: %f %c.\n", nombre, altura, metros);
System.out.printf("Mi estatura solo con 2 decimales es: %.2f", altura);
```

El código anterior imprime por pantalla:

```
Hola me llamo Paco y tengo 46 años
La estatura de PACO es: 1,825600 m.
Mi estatura solo con 2 decimales es: 1,83
```